BON VOYAGE BOT

A Conversational AI Agent for Personalized Travel Planning

PROBLEM STATEMENT

Contemporary travel planning is a fragmented, time-intensive process that requires users to navigate multiple platforms including search engines, flight aggregators, hotel booking sites, and itinerary tools that neither share context nor adapt to individual user needs. Existing solutions produce generic recommendations that fail to account for personal budget constraints, travel preferences, or trip-specific requirements. Further, once a plan is generated, modifying it requires users to restart the process entirely, creating significant friction in the overall planning process.

TARGET USER

Bon Voyage Bot is designed for casual travelers planning domestic or international trips, regardless of group size or composition. The target user is someone who lacks the time for extensive manual research, finds the current landscape of travel tools overwhelming and disjointed, and desires a planning experience that is both adaptive and conversational.

GOAL

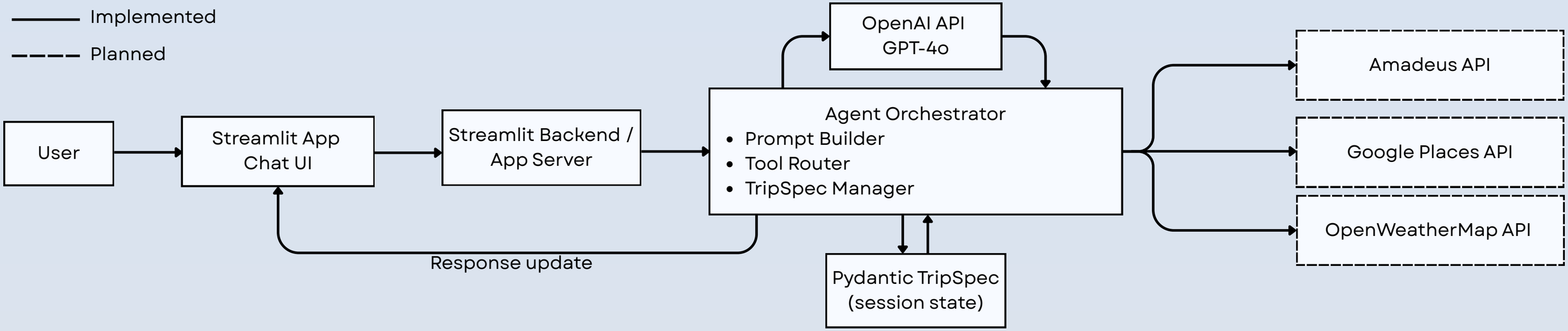
The goal of this project is to design and implement an end-to-end conversational AI travel planning assistant that enables users to describe their trip requirements in natural language and receive a personalized and budget-aware itinerary with the ability to refine their plan through dialogue. The system aims to consolidate the fragmented travel planning process into a single intelligent interface.

METHODOLOGY

PROPOSED TECHNICAL APPROACH

Bon Voyage Bot is implemented as a conversational AI agent built on GPT-4o via the OpenAI API, deployed through a Streamlit web interface. The core system design centers on an agent orchestrator that manages prompt construction, tool routing, and structured state management. User inputs are processed through a dual pipeline: GPT-4o generates conversational responses while simultaneously extracting structured trip details into a Pydantic schema, TripSpec. The Tool Router evaluates TripSpec for completeness to determine when sufficient information exists to invoke external APIs including Amadeus for flight and hotel searches, Google Places for activity recommendations, and OpenWeatherMap for weather data. This architecture enables the system to ground LLM responses in real world data while maintaining a personalized conversation.

ARCHITECTURE DIAGRAM



CODE SNIPPET I

```python
from typing import Optional, Union
from pydantic import BaseModel

class Origin(BaseModel):
    city: str = ""
    airport_code: str = ""

class Dates(BaseModel):
    start_date: Optional[str] = None
    end_date: Optional[str] = None
    month: Optional[Union[str, int]] = None
    nights: Optional[int] = None

class Budget(BaseModel):
    total_usd: Optional[float] = None
    flight_max_usd: Optional[float] = None
    lodging_per_night_max_usd: Optional[float] = None

class TripSpec(BaseModel):
    origin: Origin = Origin()
    dates: Dates = Dates()
    budget: Budget = Budget()
    must_haves: list[str] = []
    dealbreakers: list[str] = []
```

EXPLANATION OF SNIPPET I

FUNCTIONALITY

The code defines a structured, validated data schema for storing trip information extracted from the user's conversation. Using Pydantic's BaseModel, each class represents a distinct component of the trip including origin, dates, and budget, which are then composed into a single TripSpec object that persists throughout the session.

IMPORTANCE

The TripSpec schema serves as the shared memory of the entire system. Every other component depends on the information stored in it. The Agent Orchestrator reads from it to determine which APIs to call, the Tool Router evaluates its completeness before triggering external services, and the Streamlit UI uses it to render contents throughout the conversation. Without a structured schema, the system would rely entirely on unstructured conversation history, making it impossible to reliably extract and act on trip details.

PYDANTIC

Pydantic provides automatic type validation, ensuring that data extracted by GPT-4o is always clean and correctly typed before being stored. For example, if the model returns a budget as a string instead of a float, Pydantic converts it automatically, preventing downstream errors. This is particularly important in an LLM powered system where model outputs can be unpredictable or inconsistently formatted.

CODE SNIPPET 2

```
client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"))

def extract_trip_spec(messages, current_spec: TripSpec) -> TripSpec:
    try:
        response = client.chat.completions.create(
            model="gpt-4o-mini",
            messages=messages + [{"role": "system",
                                  "content": EXTRACTION_PROMPT}],
            response_format={"type": "json_object"}
        )
        extracted = json.loads(response.choices[0].message.content)
        updated = current_spec.model_dump()
        updated.update(extracted)
        return TripSpec(**updated)
    except (ValidationError, Exception):
        return current_spec
```

EXPLANATION OF SNIPPET 2

FUNCTIONALITY

This function takes the full conversation history and sends it to GPT-4o along with a structured extraction prompt. The extracted data is then merged with the existing TripSpec and validated through Pydantic before being saved to session state. If extraction fails for any reason, the function safely returns the previous spec unchanged.

IMPORTANCE

This function is what transforms a basic chatbot into an intelligent AI agent. Rather than simply responding to messages, the system is continuously building a structured understanding of the user's needs in the background. The response_format parameter forces GPT-4o to return valid JSON every time, making the extraction reliable and consistent. This structured state is what enables the Tool Router to intelligently decide when enough information has been gathered to trigger external API calls for flights, hotels, and activities.

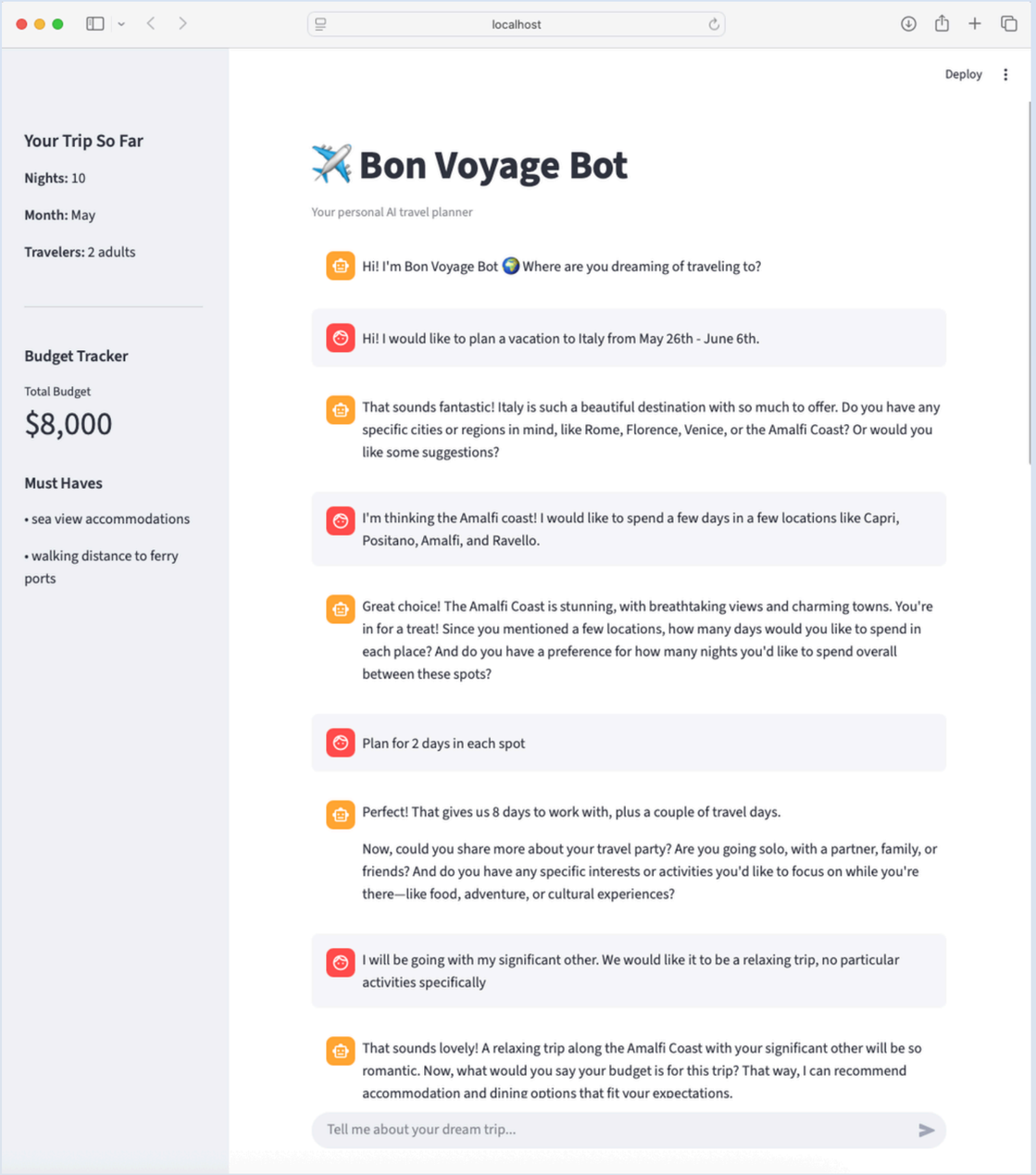
PROMPT ENGINEERING

For this project, prompt engineering serves as the bridge between unstructured natural language of conversation and the structured data that drives the agent's decision making. The function shown in the code uses a prompt that provides explicit instructions and enforces output rules. Since it is crucial for this project, I have included a section of the EXTRACTION_PROMPT below.

```
EXTRACTION_PROMPT = Today's date is {current_date}. Use this as
a reference for any travel dates mentioned. If a user mentions a
date without a year, assume it is the next upcoming occurrence.
Extract travel details and return ONLY a JSON object matching
the TripSpec structure exactly.

Rules:
- Only include fields explicitly mentioned in conversation
- Use null for any fields not mentioned
- origin.city is where the user is traveling FROM
- dates.nights is the total number of nights
- budget.total_usd is the overall trip budget in USD
- dates.month should be full month name e.g. "March"
- If no year mentioned, use the current year
- Return ONLY valid JSON, no extra text
```

PRELIMINARY RESULTS



RESULT ANALYSIS & NEXT STEPS

ANALYSIS

The preliminary result shown on the previous slide demonstrates that the core system architecture is functioning as intended. GPT-4o successfully generates contextually aware conversational responses while simultaneously extracting trip data. The sidebar located on the left side of the screen confirms that the Pydantic schema and session state management are operating correctly across multiple conversation turns.

NEXT STEPS

- Integrate external APIs including Amadeus, Google Places, and OpenWeatherMap to enable real flight, hotel, activity, and weather information.
- Implement Tool Router logic to automatically trigger API calls when sufficient trip details have been gathered
- Build an interactive Folium map visualization to render suggested locations, hotels, and activities as a day-by-day itinerary
- Determine clear success metric, such as a downloadable PDF with all travel information and list of action items for user, including flights and hotels to book
- Deploy the application to Streamlit Community Cloud for public access